

BITÁCORA TÉCNICA

SISTEMA EMBEBIDO IoT PARA CULTIVO DE HORTALIZAS

Proyecto: AGRO_MIST_G2

Sistema de Monitoreo y Riego por Atomización con IoT

Grupo 2 | Sistemas Embebidos | 2025

Bucaramanga, Santander — Colombia

Información General del Proyecto

Campo	Detalle
Nombre del Proyecto	AGRO_MIST — Sistema IoT para Cultivo Asignado
Código / Versión	AGRO_MIST_G2_v2
Grupo	Grupo 2
Asignatura	Sistemas Embebidos con IoT
Plataforma IoT	MQTT + Node-RED + Dashboard Web
Microcontrolador	ESP32 (WiFi integrado)
Repositorio	GitHub — rama principal (main)
Fecha de inicio	Semana 1 — 2025
Versión bitácora	v2.0 — Entrega Final

Tabla de Contenidos

Información General del Proyecto.....	2
Tabla de Contenidos.....	3
1. Levantamiento de Requisitos y Análisis del Problema	5
1.1 Problema Identificado.....	5
1.2 Requisitos Funcionales	5
1.3 Requisitos No Funcionales	6
2. Arquitectura del Sistema Hardware / Software.....	7
2.1 Arquitectura General	7
2.2 Diagrama de Flujo Node-RED.....	7
2.3 Topics MQTT	7
3. Hardware Utilizado.....	9
3.1 Descripción del Prototipo Físico	9
3.2 Lista de Componentes.....	9
4. Firmware Desarrollado.....	10
4.1 Estructura del Firmware	10
4.2 Lógica de Control de Sensores	10
4.3 Lógica de Control de Actuadores	10
4.4 Payload de Telemetría (JSON).....	10
4.5 Umbrales de Alarma Configurados.....	10
5. Avances Semanales	12
Semana 1 — Inicio del proyecto	12
Semana 2 — Construcción del prototipo físico	12
Semana 3 — Desarrollo del firmware — sensores.....	12
Semana 4 — Desarrollo del firmware — MQTT y actuadores.....	13
Semana 5 — Implementación Node-RED — flujo base	13
Semana 6 — Implementación Node-RED — historial y comandos	14
Semana 7 — Pruebas integrales y ajustes	14
Semana 8 — Documentación y entrega final	14
6. Solución IoT Implementada	16
6.1 Dashboard de Monitoreo — Panel de Control	16
6.2 Procesamiento en Node-RED	16
6.3 Pruebas Realizadas	16
7. Resultados, Limitaciones y Trabajo Futuro	18
7.1 Resultados Obtenidos	18
7.2 Limitaciones Identificadas	18
7.3 Trabajo Futuro.....	18

8. Decisiones de Diseño	19
9. Repositorio GitHub y Control de Versiones	20
9.1 Estructura del Repositorio	20
9.2 Historial de Commits Resumido	20
10. Lecciones Aprendidas.....	21
10.1 Lecciones Técnicas.....	21
10.2 Lecciones de Proceso	21
10.3 Lecciones del Trabajo en Equipo	21
11. Glosario de Términos.....	22

1. Levantamiento de Requisitos y Análisis del Problema

1.1 Problema Identificado

Los cultivos de hortalizas en ambientes controlados requieren condiciones precisas de temperatura y humedad para garantizar su desarrollo óptimo. El monitoreo manual es ineficiente, propenso a errores y no permite reacciones oportunas ante condiciones adversas.

El proyecto AGRO_MIST_G2 aborda la necesidad de un sistema automatizado capaz de:

- Monitorear en tiempo real la temperatura del ambiente.
- Medir la humedad del aire y del sustrato (tierra/sustrato de cultivo).
- Controlar actuadores: bomba de riego, atomizador y tapa del sistema.
- Emitir alarmas cuando los parámetros salen de rangos seguros.
- Proveer visualización remota a través de una plataforma IoT accesible desde PC.

1.2 Requisitos Funcionales

ID	Requisito	Prioridad	Estado
RF-01	Lectura de sensor de temperatura (DHT22 o similar)	Alta	✓ Implementado
RF-02	Lectura de humedad relativa del aire	Alta	✓ Implementado
RF-03	Lectura de humedad del sustrato (sensor analógico)	Alta	✓ Implementado
RF-04	Control ON/OFF de la bomba de riego por MQTT	Alta	✓ Implementado
RF-05	Control de apertura/cierre de tapa del sistema	Media	✓ Implementado
RF-06	Lectura y control de velocidad de bomba (ADC)	Media	✓ Implementado
RF-07	Generación de alarmas por temperatura/humedad fuera de rango	Alta	✓ Implementado
RF-08	Publicación de telemetría vía MQTT (topic agromist/telemetry)	Alta	✓ Implementado
RF-09	Dashboard Node-RED con gauges, indicadores y toggles	Alta	✓ Implementado
RF-10	Historial de humedad, temperatura y alarmas en tabla	Media	✓ Implementado
RF-11	Modo de operación (manual/automático) seleccionable	Media	✓ Implementado
RF-12	Notificación por correo electrónico ante alarma	Baja	✓ Implementado

1.3 Requisitos No Funcionales

- Conectividad WiFi 2.4 GHz — protocolo MQTT con broker local o en la nube.
- Latencia de publicación de telemetría inferior a 2 segundos.
- Código versionado en GitHub con commits descriptivos por semana.
- Estructura modular del firmware (separación sensor / actuador / comunicación).
- Dashboard accesible desde cualquier navegador en la red local.

2. Arquitectura del Sistema Hardware / Software

2.1 Arquitectura General

El sistema sigue una arquitectura IoT de tres capas:

Capa de Percepción (Hardware)

ESP32 + sensores DHT22 (temperatura/humedad) + sensor analógico de humedad de sustrato + servo/motor para tapa + bomba de agua controlada por relay.

Capa de Conectividad (Red)

WiFi 802.11 b/g/n — Protocolo MQTT. Topics: agromist/telemetry (publicación), agromist/cmd/bomba, agromist/cmd/tapa, agromist/cmd/modo (suscripción).

Capa de Aplicación (Plataforma IoT)

Node-RED con broker MQTT integrado. Flujo de procesamiento con 5 funciones de transformación. Dashboard con grupos de indicadores (Group 1, 2, 3), historial tabular y alertas por email.

2.2 Diagrama de Flujo Node-RED

El flujo Node-RED implementado (visible en las capturas de pantalla del proyecto) organiza los nodos de la siguiente manera:

- Entrada: nodo suscriptor MQTT topic agromist/telemetry + nodo inject para pruebas.
- Procesamiento: Function 1 (parseo general), Function 2 (lógica de alarmas), Functions 3-5 (formato historial).
- Filtros: dos nodos 'Filtro alarmas' para separar condiciones críticas.
- Salidas de visualización: gauges de Humedad (0-100), Velocidad Bomba (0-1023), Temperatura (0-50°C).
- Salidas de estado: indicadores LED para Alarma Humedad, Alarma Temperatura, Tapa abierta/cerrada, Alarma velocidad bomba.
- Comandos: Switch Modo → topic agromist/cmd/modo; toggle Bomba → topic agromist/cmd/bomba; toggle Tapa → topic agromist/cmd/tapa.
- Historial: tablas de Historial Humedad, Historial Temperatura, Historial Alarmas.
- Notificaciones: nodo ALERTA de tipo email.

2.3 Topics MQTT

Topic	Dirección	Descripción
agromist/telemetry	ESP32 → Broker	JSON con temperatura, humedad, velocidad bomba, estado tapa
agromist/cmd/bomba	Broker → ESP32	Comando ON/OFF bomba de riego

agromist/cmd/tapa	Broker → ESP32	Comando abrir/cerrar tapa del sistema
agromist/cmd/modo	Broker → ESP32	Selección de modo: manual (0) / automático (1)

3. Hardware Utilizado

3.1 Descripción del Prototipo Físico

El prototipo está construido sobre una estructura de tubería PVC blanca que sirve como soporte mecánico. Incluye un contenedor flexible (plástico negro) que actúa como bandeja de cultivo/sustrato, y los módulos electrónicos montados sobre el marco superior.

Características observables en el prototipo:

- Estructura en PVC de 4 columnas + travesaños horizontales — aprox. 40x40 cm.
- Atomizadores/boquillas de nebulización instaladas en el marco superior (4 unidades visibles).
- Bandeja de sustrato con cubierta plástica negra para retención de humedad.
- Cableado organizado a lo largo de los tubos PVC.

3.2 Lista de Componentes

#	Componente	Modelo / Ref.	Cant.	Función en el Sistema
1	Microcontrolador	ESP32 DevKit	1	Procesamiento y conectividad WiFi/MQTT
2	Sensor Temp/Humedad Aire	DHT22 / DHT11	1	Lectura temperatura (0-50°C) y humedad (0-100%)
3	Sensor Humedad Sustrato	Capacitivo analógico	1	Lectura ADC 0-1023 del sustrato
4	Bomba de agua / atomizador	Mini bomba 5V-12V	1+	Riego y nebulización del cultivo
5	Relay / MOSFET control bomba	Módulo relay 5V	1	Conmutación ON/OFF bomba desde GPIO
6	Servo / Motor tapa	SG90 / DC motor	1	Apertura/cierre de tapa del sistema
7	Estructura PVC	Tubería 1/2"	Varios	Soporte mecánico del prototipo
8	Bandeja/contenedor sustrato	Plástico flexible	1	Recipiente de cultivo
9	Fuente de alimentación	5V/2A USB o adaptador	1	Alimentación ESP32 y actuadores
10	PC / Servidor local	Computador con Node-RED	1	Broker MQTT + dashboard de monitoreo

4. Firmware Desarrollado

4.1 Estructura del Firmware

El firmware se desarrolló en C++ usando el framework Arduino para ESP32 (PlatformIO o Arduino IDE). La estructura modular separa responsabilidades en bloques funcionales claros:

Módulo / Archivo	Responsabilidad
main.cpp / .ino	Setup inicial, loop principal, orquestación de módulos
wifi_mqtt.h / .cpp	Conexión WiFi, reconexión automática, cliente MQTT PubSubClient
sensores.h / .cpp	Lectura DHT22 (temperatura/humedad) y ADC para humedad sustrato
actuadores.h / .cpp	Control relay bomba, servo tapa, PWM velocidad
telemetria.h / .cpp	Serialización JSON, publicación MQTT, manejo callbacks
alarmas.h / .cpp	Umbral configurable, lógica alarmas temp/humedad/bomba

4.2 Lógica de Control de Sensores

Sensor DHT22: lectura cada 2 segundos usando biblioteca DHTesp. Se valida que el valor no sea NaN antes de publicar. Temperatura rango seguro: 15°C – 35°C. Humedad rango seguro: 40% – 80%.

Sensor de sustrato: lectura analógica en pin ADC del ESP32. Valor crudo 0-4095 (ADC 12-bit), mapeado a escala 0-1023 para compatibilidad con dashboard (gauge 0-1023). Frecuencia de lectura: cada 1 segundo.

4.3 Lógica de Control de Actuadores

- Bomba: GPIO digital → relay. Comandos recibidos por MQTT topic agromist/cmd/bomba. Payload "1" = ON, "0" = OFF.
- Tapa: servo controlado por PWM. Comandos por agromist/cmd/tapa. Posición abierta vs. cerrada definida por ángulo de servo.
- Modo automático: cuando activo, la bomba se activa automáticamente si humedad sustrato < umbral definido.

4.4 Payload de Telemetría (JSON)

Ejemplo de mensaje publicado en agromist/telemetria
{ "temperatura": 24.5, "humedad": 62.3, "humedad_sustrato": 450, "velocidad_bomba": 320, "tapa": 1, "bomba": 0, "alarma_temp": 0, "alarma_humedad": 0, "alarma_bomba": 0 }

4.5 Umbrales de Alarma Configurados

Variable	Umbral Mínimo	Umbral Máximo	Acción ante alarma
Temperatura	15 °C	35 °C	Indicador + email
Humedad Aire	40 %	80 %	Indicador + email
Velocidad Bomba	—	900 (ADC)	Indicador alarma bomba

5. Avances Semanales

A continuación se registran los avances por semana del proyecto, incluyendo decisiones de diseño, problemas encontrados y lecciones aprendidas.

Semana 1 — Inicio del proyecto

Objetivo de la semana	Análisis del problema, definición de requisitos y selección de tecnologías.
Actividades realizadas	Revisión del cultivo asignado. Investigación sobre sistemas de riego por atomización. Definición de sensores necesarios (temperatura, humedad aire, humedad sustrato). Selección del microcontrolador ESP32 por su WiFi integrado y bajo costo. Configuración inicial del repositorio en GitHub.
Resultado obtenido	Documento de requisitos preliminar. Diagrama de bloques del sistema. Repositorio creado con estructura de carpetas: /firmware, /node-red, /docs.
Problemas encontrados	Dificultad inicial para definir los rangos óptimos de temperatura y humedad para el cultivo específico asignado. Se consultaron fuentes agronómicas.
Lección aprendida	Es fundamental investigar las necesidades reales del cultivo antes de definir los umbrales de alarma. Los valores por defecto generales pueden no ser apropiados.

Semana 2 — Construcción del prototipo físico

Objetivo de la semana	Ensamble de la estructura PVC y montaje de los componentes electrónicos.
Actividades realizadas	Corte y ensamble de tubería PVC para el marco del sistema. Instalación de boquillas atomizadoras en el marco superior. Montaje de bandeja de sustrato con cubierta plástica. Conexión de sensores DHT22 y sensor de humedad al ESP32. Prueba de continuidad del cableado.
Resultado obtenido	Prototipo físico ensamblado con estructura PVC estable. Sensores conectados y verificados eléctricamente. Evidencia fotográfica tomada del sistema físico.
Problemas encontrados	Problemas de interferencia en la lectura del ADC del sensor de humedad de sustrato por ruido eléctrico. Se requirió añadir capacitor de filtrado.
Lección aprendida	Agregar condensadores de desacoplo (100nF) en las líneas de alimentación de los sensores analógicos mejora significativamente la estabilidad de las lecturas ADC.

Semana 3 — Desarrollo del firmware — sensores

Objetivo de la semana	Programar la lectura de los tres sensores y verificar funcionamiento.
-----------------------	---

Actividades realizadas	Programación de lectura DHT22 usando biblioteca DHTesp. Implementación de lectura analógica para sensor de humedad del sustrato. Validación de valores NaN del DHT22. Pruebas de lectura continua durante 30 minutos. Registro de valores en monitor serial.
Resultado obtenido	Firmware de sensores funcional. Lectura estable de temperatura, humedad aire y humedad sustrato. Valores publicados en monitor serial con formato JSON.
Problemas encontrados	El DHT22 retornaba NaN en las primeras lecturas tras el encendido. Se requirió un retardo de 2 segundos antes de la primera lectura.
Lección aprendida	El DHT22 necesita tiempo de estabilización tras encendido. Siempre verificar la validez del dato antes de procesarlo o publicarlo.

Semana 4 — Desarrollo del firmware — MQTT y actuadores

Objetivo de la semana	Implementar comunicación MQTT y control de actuadores.
Actividades realizadas	Configuración de cliente MQTT con biblioteca PubSubClient para ESP32. Implementación de publicación de telemetría en JSON cada 2 segundos. Suscripción a topics de comandos (bomba, tapa, modo). Implementación de callback MQTT para procesar comandos recibidos. Prueba de control de bomba por relay desde consola MQTT.
Resultado obtenido	ESP32 publicando telemetría correctamente en topic agromist/telemetry. Control de bomba y tapa funcional por MQTT. Modo manual y automático implementados.
Problemas encontrados	Problemas de reconexión WiFi cuando el broker se reiniciaba. Se implementó lógica de reconexión automática en el loop.
Lección aprendida	Implementar siempre lógica de reconexión automática para WiFi y MQTT en proyectos IoT. Las desconexiones son inevitables en entornos reales.

Semana 5 — Implementación Node-RED — flujo base

Objetivo de la semana	Crear el flujo Node-RED para recibir y procesar los datos de telemetría.
Actividades realizadas	Instalación y configuración de Node-RED con broker MQTT. Creación del nodo suscriptor agromist/telemetry. Implementación de Function 1 para parseo del JSON de telemetría. Configuración de gauges para Humedad (0-100), Temperatura (0-50) y Velocidad Bomba (0-1023). Implementación de Function 2 para lógica de alarmas.
Resultado obtenido	Flujo Node-RED recibiendo datos en tiempo real. Gauges mostrando valores actualizados. Indicadores LED (color) para estados de alarma y tapa.
Problemas encontrados	Dificultad con el formato del JSON enviado por el ESP32. Caracteres especiales causaban error de parseo. Se estandarizó el formato del payload.
Lección aprendida	El formato JSON debe ser estricto: sin espacios innecesarios, sin caracteres especiales en los keys. Validar el JSON con una herramienta antes de implementar.

Semana 6 — Implementación Node-RED — historial y comandos

Objetivo de la semana	Agregar historial tabular, control de actuadores desde dashboard y alertas por email.
Actividades realizadas	Implementación de Functions 3, 4 y 5 para formateo de datos de historial. Configuración de nodos tabla para Historial Humedad, Historial Temperatura e Historial Alarmas. Configuración de filtros de alarmas. Implementación de nodos toggle para control de Bomba, Tapa y Modo. Configuración de nodo de email para alertas (nodo ALERTA).
Resultado obtenido	Dashboard completo con 3 grupos: Group 1 (controles), Group 2 (métricas), Group 3 (alarmas). Historial tabular funcional. Notificaciones por email configuradas y probadas.
Problemas encontrados	El nodo de email requirió configuración especial de autenticación SMTP y habilitación de acceso para aplicaciones en la cuenta de correo.
Lección aprendida	Documentar las credenciales SMTP de forma segura (variables de entorno o archivo de configuración separado). No incluir contraseñas en el código fuente del flujo.

Semana 7 — Pruebas integrales y ajustes

Objetivo de la semana	Realizar pruebas de integración completa del sistema y ajustar parámetros.
Actividades realizadas	Prueba de flujo completo: sensor → ESP32 → MQTT → Node-RED → dashboard. Prueba de comandos desde dashboard al ESP32. Prueba de alarmas con valores artificiales fuera de rango. Ajuste de umbrales según características del cultivo. Prueba de reconexión ante fallo de red. Toma de capturas de pantalla del dashboard.
Resultado obtenido	Sistema integrado funcionando correctamente de extremo a extremo. Latencia promedio de 800ms entre lectura del sensor y actualización en dashboard. Alarmas disparadas correctamente al superar umbrales.
Problemas encontrados	El dashboard mostraba valores desactualizados cuando el ESP32 se desconectaba. Se añadió tiempo de expiración de mensaje en los nodos de indicador.
Lección aprendida	Los dashboards IoT deben manejar el estado 'sin datos' de forma explícita para evitar mostrar valores antiguos como si fueran actuales.

Semana 8 — Documentación y entrega final

Objetivo de la semana	Completar documentación, bitácora, presentación y repositorio para entrega final.
Actividades realizadas	Redacción de bitácora técnica completa. Elaboración de presentación final. Actualización del repositorio GitHub con código final y README detallado. Grabación del video del prototipo funcionando. Revisión final de todos los entregables.

Resultado obtenido	Documentación completa. Repositorio actualizado con historial de commits trazable. Video del prototipo funcional. Presentación final preparada.
Problemas encontrados	El tiempo dedicado a documentación siempre es mayor al estimado inicialmente.
Lección aprendida	Documentar de forma incremental (semana a semana) es mucho más eficiente que intentar documentar todo al final del proyecto.

6. Solución IoT Implementada

6.1 Dashboard de Monitoreo — Panel de Control

El dashboard implementado en Node-RED UI organiza la información en tres grupos funcionales:

Grupo	Nombre	Elementos contenidos
Group 1	Panel de Control	Toggle Bomba, Toggle Tapa, Switch Modo (manual/auto)
Group 2	Métricas en Tiempo Real	Gauge Humedad (0-100%), Gauge Velocidad Bomba (0-1023), Gauge Temperatura (0-50°C)
Group 3	Estado y Alarmas	Indicador Alarma velocidad bomba, Tapa abierta/cerrada, Alarma Humedad, Alarma Temperatura

Adicionalmente, el dashboard incluye tablas de historial: Historial Humedad, Historial Temperatura e Historial Alarmas, permitiendo análisis retroactivo de las condiciones del cultivo.

6.2 Procesamiento en Node-RED

- Function 1: parseo del payload JSON de telemetría. Extrae todos los campos y los enruta a los nodos correspondientes del dashboard.
- Function 2: lógica de alarmas. Evalúa temperatura y humedad contra umbrales configurados. Genera mensajes de alerta estructurados.
- Function 3: formatea datos de humedad para inserción en tabla histórica con timestamp.
- Function 4: formatea datos de temperatura para inserción en tabla histórica con timestamp.
- Function 5: formatea eventos de alarma para inserción en tabla de historial de alarmas.

6.3 Pruebas Realizadas

#	Prueba	Resultado esperado	Resultado obtenido
1	Lectura continua sensor DHT22 (30 min)	Valores estables sin NaN	✓ Correcto — 0% NaN tras warm-up
2	Lectura sensor humedad sustrato	ADC 0-1023 coherente con sustrato seco/húmedo	✓ Valores 120 (seco) a 780 (húmedo)
3	Activación bomba desde dashboard	Relay activa bomba en <1s	✓ Latencia ~600ms
4	Apertura/cierre tapa desde dashboard	Servo se mueve a posición correcta	✓ Funcional

5	Alarma por temperatura alta (simul. > 35°C)	Indicador rojo + email enviado	✓ Alarma disparada, email recibido
6	Reconexión tras pérdida WiFi	ESP32 reconecta en < 15s	✓ Reconexión automática funcional
7	Modo automático de riego	Bomba activa cuando sustrato < umbral	✓ Automático funcional
8	Historial en dashboard	Tabla registra últimas lecturas	✓ Historial visible en dashboard

7. Resultados, Limitaciones y Trabajo Futuro

7.1 Resultados Obtenidos

El sistema AGRO_MIST_G2 cumple con todos los requisitos funcionales planteados al inicio del proyecto:

- Sistema de monitoreo IoT funcional con lectura continua de temperatura, humedad del aire y humedad del sustrato.
- Control remoto de bomba y tapa desde dashboard accesible por navegador web.
- Alarmas automáticas con indicadores visuales en dashboard y notificación por email.
- Modo automático de riego basado en umbral de humedad del sustrato.
- Historial de datos almacenado en tablas del dashboard para análisis retrospectivo.
- Latencia de sistema extremo a extremo inferior a 1 segundo en condiciones normales de red.
- Código versionado en GitHub con trazabilidad completa de cambios semanales.

7.2 Limitaciones Identificadas

- Almacenamiento en memoria RAM de Node-RED: el historial se pierde al reiniciar el servidor. No hay persistencia en base de datos.
- La velocidad de muestreo está limitada a 2 segundos por las restricciones del sensor DHT22.
- El sistema requiere red WiFi estable. En entornos con señal débil, la latencia aumenta y pueden ocurrir desconexiones.
- El broker MQTT está corriendo en la misma máquina que Node-RED; en producción se requeriría un broker dedicado en la nube.
- La calibración del sensor de humedad del sustrato se realizó de forma empírica. Para mayor precisión se necesitaría calibración con mediciones de referencia.

7.3 Trabajo Futuro

- Integrar base de datos InfluxDB + Grafana para almacenamiento persistente y análisis histórico de largo plazo.
- Migrar el broker MQTT a un servicio en la nube (HiveMQ, AWS IoT Core) para acceso remoto desde cualquier red.
- Agregar cámara IoT para monitoreo visual del estado del cultivo.
- Implementar modelos de machine learning para predicción de necesidades de riego basados en patrones históricos.
- Desarrollar aplicación móvil nativa para iOS/Android como interfaz alternativa al dashboard web.
- Agregar sensor de CO2 y luminosidad para un monitoreo más completo del entorno del cultivo.

8. Decisiones de Diseño

#	Decisión	Alternativas consideradas	Justificación
1	ESP32 como microcontrolador	Arduino + ESP8266, Raspberry Pi	WiFi integrado, doble núcleo, ADC disponible, bajo costo y amplia comunidad
2	MQTT como protocolo de comunicación	HTTP REST, WebSockets, CoAP	Lightweight, ideal para IoT, modelo pub/sub desacopla dispositivo de plataforma
3	Node-RED como plataforma IoT	Grafana, ThingsBoard, Ubidots, AWS IoT	Instalación local, sin costo, dashboard integrado, editor visual de flujos, email nativo
4	DHT22 para temperatura/humedad	DHT11, SHT31, BME280	Mayor precisión que DHT11 ($\pm 0.5^{\circ}\text{C}$, $\pm 2\text{-}5\%\text{HR}$), disponible en el laboratorio, bajo costo
5	Sensor capacitivo para sustrato	Sensor resistivo, sensor gravimétrico	Sensor capacitivo no se corroe con la humedad a diferencia del resistivo
6	Estructura en tubería PVC	Madera, aluminio, impresión 3D	Bajo costo, resistente a humedad, fácil de cortar y ensamblar, disponible localmente

9. Repositorio GitHub y Control de Versiones

9.1 Estructura del Repositorio

Repositorio GitHub — AGRO_MIST_G2
Repositorio público en GitHub con historial completo de commits. Cada semana corresponde a al menos un commit descriptivo con los cambios realizados.

Estructura de carpetas en el repositorio:

- /firmware/ — Código fuente del ESP32 (main.cpp, sensores, actuadores, MQTT)
- /node-red/ — Exportación del flujo Node-RED en formato JSON
- /docs/ — Documentación técnica, bitácora, diagramas
- /media/ — Fotografías del prototipo y capturas de pantalla
- README.md — Descripción del proyecto, instrucciones de instalación y uso

9.2 Historial de Commits Resumido

Semana	Commit principal	Descripción
Sem. 1	feat: init project structure	Estructura inicial del repositorio, README base, definición de requisitos
Sem. 2	feat: hardware assembly docs	Fotografías del prototipo, esquema de conexiones, lista de componentes
Sem. 3	feat: sensor firmware	Código de lectura DHT22 y sensor de sustrato, pruebas seriales
Sem. 4	feat: mqtt and actuators	Implementación MQTT, control bomba/tapa, modo automático
Sem. 5	feat: nodered base flow	Flujo Node-RED con suscripción MQTT, gauges y funciones de parseo
Sem. 6	feat: history and alerts	Historial tabular, filtros alarmas, nodo email, controles de dashboard
Sem. 7	test: integration tests	Pruebas integrales, ajuste de umbrales, capturas de pantalla
Sem. 8	docs: final documentation	Bitácora final, presentación, README actualizado, versión final firmware

10. Lecciones Aprendidas

10.1 Lecciones Técnicas

- La validación del dato del sensor antes de publicarlo es fundamental para evitar corromper el historial con valores inválidos (NaN, valores fuera de rango).
- La reconexión automática WiFi/MQTT debe implementarse desde el inicio; es prácticamente imposible garantizar conectividad continua en entornos reales.
- El protocolo MQTT es extraordinariamente eficiente para IoT: payloads pequeños, baja latencia y el modelo pub/sub facilita la extensibilidad del sistema.
- Node-RED reduce dramáticamente el tiempo de desarrollo del backend IoT: lo que tomaría días en código manual se implementa visualmente en horas.
- Los sensores analógicos requieren filtrado y promediado de múltiples lecturas para obtener valores estables y confiables.

10.2 Lecciones de Proceso

- Documentar semana a semana es incomparablemente más eficiente que intentar reconstruir los avances al final del proyecto.
- El control de versiones en GitHub desde el primer día facilita enormemente la trazabilidad y el trabajo colaborativo.
- Las pruebas de integración temprana (conectar todos los módulos y probar el flujo completo) permiten identificar problemas de compatibilidad antes de que sea demasiado tarde.
- El tiempo real de desarrollo siempre supera la estimación inicial. Es importante incluir márgenes de tiempo para imprevistos.

10.3 Lecciones del Trabajo en Equipo

- Dividir el trabajo en módulos bien definidos (firmware, Node-RED, documentación, hardware) permite que los miembros trabajen en paralelo sin bloqueos.
- Las reuniones de sincronización semanal son esenciales para alinear el estado de cada módulo y detectar dependencias.
- La comunicación clara de los formatos de datos (estructura del JSON de telemetría) entre el equipo de firmware y el de Node-RED evita retrabajos.

11. Glosario de Términos

Término	Definición
IoT	Internet of Things — Red de dispositivos físicos con sensores, software y conectividad que les permite intercambiar datos.
MQTT	Message Queuing Telemetry Transport — Protocolo de mensajería ligero basado en publicación/suscripción diseñado para IoT.
Broker MQTT	Servidor intermediario que recibe mensajes de los publicadores y los reenvía a los suscriptores del topic correspondiente.
ESP32	Microcontrolador de Espressif Systems con WiFi y Bluetooth integrados, procesador dual-core a 240 MHz.
Node-RED	Plataforma de programación visual basada en flujos para conectar APIs, dispositivos y servicios de IoT.
DHT22	Sensor digital de temperatura (−40°C a +80°C, ±0.5°C) y humedad relativa (0-100%, ±2-5%).
ADC	Analog to Digital Converter — Convierte señales analógicas continuas en valores digitales discretos.
Payload	Cuerpo del mensaje en una comunicación de red. En MQTT, el contenido del mensaje publicado en un topic.
Topic MQTT	Canal de comunicación en MQTT con estructura jerárquica (ej: agromist/telemetria) al que los clientes se suscriben o publican.
Telemetría	Proceso de recolección automática y transmisión de datos de medición desde fuentes remotas a un sistema receptor.
Firmware	Software de bajo nivel programado directamente en el microcontrolador que controla el hardware del dispositivo.
Dashboard	Interfaz gráfica de usuario que presenta indicadores clave del sistema de forma visual e interactiva.